

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica – Scienza e Ingegneria

Corso di Laurea Magistrale in
Informatica

PoS tagging italiano con BERT

Progetto di Natural Language Processing

Studente: Pierre Errico

Matricola: 0001235580

Anno Accademico 2026/2027

Abstract

Il progetto affronta il fine-tuning di BERT italiano per il PoS tagging sul benchmark UD Italian-ISDT 2.15, con l'obiettivo di valutarne le prestazioni rispetto a UDPipe 2 e ai risultati della letteratura. Il compito principale consiste nella predizione dei tag UPOS a partire da parole e confini di frase gold, mantenendo gli split ufficiali del corpus e confrontando i sistemi mediante accuracy, macro-F1 e analisi degli errori. Per approfondire il contributo dell'adattamento del modello e della decodifica strutturata, gli esperimenti comprendono un encoder congelato, BERT fine-tuned e una variante con CRF, valutati su tre seed sia sul solo UPOS sia dopo l'estensione alla predizione delle caratteristiche morfologiche `FEATS`.

BERT fine-tuned raggiunge un'accuracy UPOS di circa il 98,81%, rispetto al 98,75% di UDPipe, e dopo l'estensione morfologica arriva al 98,84%, con una macro-F1 sulle classi presenti del 95,57%. Questi risultati sono in accordo con la letteratura e indicano che il fine-tuning costituisce una soluzione efficace per il compito, mentre i confronti interni mostrano benefici del CRF soprattutto sulle categorie rare. Anche la predizione morfologica raggiunge un'accuracy esatta vicina a quella del riferimento, ma la macro-F1 sulle coppie attributo-valore evidenzia maggiori difficoltà sulle caratteristiche poco frequenti; l'analisi degli errori permette quindi di affiancare alla correttezza complessiva una lettura delle prestazioni sulle singole classi.

Indice

1 Introduzione **1**
- 1.1 Sistemi e convenzioni per l'analisi delle categorie grammaticali 1
- 1.2 Le principali sfide del PoS tagging 3
- 1.3 Modelli neurali per il linguaggio 3
- 1.4 Architettura BERT 4
  - 1.4.1 Tokenizzazione 4
  - 1.4.2 Struttura dell'encoder 4
  - 1.4.3 Preaddestramento 6
- 1.5 Obiettivi del progetto 6

2 Metodologia **7**
- 2.1 Definizione dell'esperimento 7
- 2.2 Dati e annotazioni 8
  - 2.2.1 Corpus e suddivisione 8
  - 2.2.2 Rappresentazione di UPOS e FEATS 9
- 2.3 Preprocessing e allineamento 9
- 2.4 Modelli e funzione obiettivo 9
- 2.5 Procedura di addestramento 10
- 2.6 Selezione del checkpoint 10
- 2.7 Valutazione 11
  - 2.7.1 Metriche UPOS 12
  - 2.7.2 Metriche FEATS 12
  - 2.7.3 Aggregazione e analisi degli errori 13
- 2.8 Confronto con UDPipe 2 13
- 2.9 Riproducibilità 13

3 Risultati **14**
- 3.1 Confronto UPOS con UDPipe 2 14
- 3.2 Posizionamento rispetto alla letteratura 16
- 3.3 Confronti interni tra le configurazioni BERT 17
- 3.4 Estensione alla predizione `FEATS` 17
- 3.5 Analisi degli errori 19
  - 3.5.1 Esempi contestuali 20

4 Conclusioni **21**

Bibliografia **22**

Capitolo 1

Introduzione

1.1 Sistemi e convenzioni per l'analisi delle categorie grammaticali

Nella linguistica computazionale, il Part-of-Speech (PoS) tagging è un compito di etichettatura di sequenze di token, nel quale a ogni token viene assegnata un'etichetta che ne identifica la categoria grammaticale nel contesto della frase. Un token corrisponde generalmente a una parola; può tuttavia indicare anche altri elementi linguistici, come simboli e punteggiatura. Il PoS tagging riveste un ruolo fondamentale per l'analisi linguistica e costituisce una fase preliminare per numerose applicazioni di Natural Language Processing (NLP), tra cui la lemmatizzazione, l'analisi sintattica e la traduzione automatica [1].

Universal Dependencies (UD) è un framework per l'annotazione linguistica che definisce convenzioni condivise per la rappresentazione delle categorie grammaticali, delle caratteristiche morfologiche e delle relazioni sintattiche in numerose lingue, tra cui l'italiano [2]. La standardizzazione delle annotazioni e la disponibilità di corpora multilingue ne favoriscono l'impiego nella ricerca e nello sviluppo di modelli NLP.

In particolare, UD definisce un inventario di 17 categorie grammaticali universali, denominate Universal Part-of-Speech (UPOS) tags [3]. Le caratteristiche morfologiche, come genere, numero e tempo verbale, sono rappresentate separatamente, quando applicabili, mediante gli attributi del campo `FEATS` [4]. La Tabella 1.1 riporta le categorie UPOS, accompagnate da esempi contestuali in italiano e dagli attributi FEATS attestati nel treebank UD Italian-ISDT v2.15. Questi ultimi comprendono tutti gli attributi presenti almeno una volta nelle annotazioni di ciascuna categoria, anche in casi rari, e non sono pertanto necessariamente applicabili a tutti i token appartenenti alla stessa categoria.

Tabella 1.1: Categorie UPOS, esempi contestuali in italiano e attributi morfologici FEATS attestati nel treebank UD Italian-ISDT v2.15. Il trattino indica l'assenza di attributi attestati o di un esempio contestuale. Per `PART` non è riportato un esempio in italiano, poiché nel corpus il tag è utilizzato per il marcatore possessivo inglese.

Tag UPOS	**Categoria grammaticale**	**Esempio contestuale**	**FEATS attestati**
`ADJ`	Aggettivo	Il gatto è **nero**.	`Degree`, `Gender`, `Number`, `NumType`, `Poss`, `PronType`, `Typo`
`ADP`	Adposizione	Vado **a** casa.	`Gender`, `Number`
`ADV`	Avverbio	Il gatto dorme **tranquillamente**.	`Degree`, `Gender`, `Number`, `PronType`
`AUX`	Verbo ausiliare	Maria **ha** mangiato.	`Gender`, `Mood`, `Number`, `Person`, `Tense`, `VerbForm`
`CCONJ`	Congiunzione coordinante	Leggo **e** scrivo.	–
`DET`	Determinante	**Il** gatto dorme.	`Definite`, `Gender`, `Number`, `Poss`, `PronType`
`INTJ`	Interiezione	**Oh**, sei arrivato!	`Polarity`
`NOUN`	Nome comune	Il **gatto** dorme.	`Foreign`, `Gender`, `Number`, `Tense`, `VerbForm`
`NUM`	Numerale	Ho **tre** libri.	`NumType`
`PART`	Particella	–	–
`PRON`	Pronome	**Lei** legge un libro.	`Clitic`, `Gender`, `Number`, `Person`, `Poss`, `PronType`
`PROPN`	Nome proprio	**Maria** legge un libro.	`Foreign`, `Gender`, `Number`, `NumType`
`PUNCT`	Punteggiatura	Sei a casa**?**	–
`SCONJ`	Congiunzione subordinante	Resto a casa **perché** piove.	`PronType`
`SYM`	Simbolo	Il risultato è 2 + 3.	–
`VERB`	Verbo	Il gatto **dorme**.	`Gender`, `Mood`, `Number`, `Person`, `Tense`, `VerbForm`
`X`	Altro	Ha pronunciato **xfgh**.	`Foreign`, `Gender`, `Number`

A titolo di esempio, si consideri la frase «*Il gatto dorme.*». Nell'ambito del PoS tagging, ai quattro token viene assegnata rispettivamente la sequenza di etichette

`DET NOUN VERB PUNCT`. A queste annotazioni si aggiungono, quando applicabili, le informazioni morfologiche rappresentate nel campo `FEATS`. Il token «*gatto*», ad esempio, è classificato come `NOUN` e presenta gli attributi `Gender=Masc|Number=Sing`, che ne specificano il genere maschile e il numero singolare.

1.2 Le principali sfide del PoS tagging

Una delle principali difficoltà del PoS tagging è rappresentata dall'ambiguità grammaticale: una stessa parola può appartenere a più categorie, e la scelta del tag corretto dipende dal contesto [1]. Ad esempio, la parola *porta* può essere un nome («*La porta è chiusa.*») o un verbo («*Lui porta il libro.*»).

Un altro problema risiede nello sbilanciamento della distribuzione dei tag: alcune categorie, come nomi comuni (`NOUN`), determinanti (`DET`) e adposizioni (`ADP`), sono molto frequenti, mentre altre, come simboli (`SYM`) ed elementi non classificabili nelle altre categorie (`X`), compaiono raramente [5]. La presenza di classi con pochi esempi può rendere più difficile l'apprendimento di modelli robusti e generalizzabili [6].

A ciò si aggiungono ulteriori problematiche:

- la presenza di parole rare o non osservate durante l'addestramento, tra cui neologismi, termini specialistici e nomi propri, che possono risultare difficili da classificare. Alcune di queste possono inoltre essere assenti dal vocabolario del modello (Out-of-Vocabulary, OOV) [7];
- le differenze tra il dominio dei testi di addestramento e quello dei testi da analizzare, che possono ridurre la capacità di generalizzazione del modello [1];
- la predizione degli attributi morfologici del campo `FEATS`, che introduce ulteriori difficoltà dovute alla presenza di attributi applicabili soltanto a determinate categorie grammaticali e alla distribuzione sbilanciata dei relativi valori;
- gli eventuali errori o incoerenze nelle annotazioni, che possono influenzare sia l'apprendimento sia la valutazione [1].

Queste problematiche motivano l'impiego di modelli capaci di costruire rappresentazioni linguistiche sensibili al contesto e di sfruttare conoscenze acquisite attraverso il preaddestramento su grandi quantità di dati testuali [8].

1.3 Modelli neurali per il linguaggio

L'evoluzione delle tecniche di NLP ha progressivamente migliorato la modellazione del contesto, passando da rappresentazioni lessicali statiche ad architetture in grado di integrare informazioni contestuali sempre più complesse [9].

Tra gli approcci neurali più influenti, Word2Vec [10] ha permesso di apprendere rappresentazioni vettoriali dense delle parole, dette *word embeddings*, nelle quali termini utilizzati in contesti simili tendono ad assumere rappresentazioni vicine.

Tali embedding sono tuttavia statici, ossia associano a ciascuna parola un unico vettore indipendentemente dal contesto, senza distinguere i diversi significati o ruoli grammaticali che essa può assumere.

Le reti neurali ricorrenti (RNN) producono rappresentazioni dipendenti dal contesto attraverso l'elaborazione sequenziale dei token. Le Long Short-Term Memory (LSTM) migliorano la capacità di modellare dipendenze a lungo termine, mentre le varianti bidirezionali, come le BiLSTM, integrano informazioni provenienti da entrambe le direzioni della sequenza [9, 7]. La natura ricorrente di queste architetture limita tuttavia la parallelizzazione e può rendere difficile la modellazione di dipendenze particolarmente distanti [11].

L'architettura Transformer [11] affronta alcuni di questi limiti sostituendo la ricorrenza con meccanismi di attenzione, tra cui la *self-attention*, che permette di modellare direttamente le relazioni tra i token, anche distanti tra loro, favorendo un'elaborazione parallela della sequenza e la produzione di rappresentazioni contestuali.

1.4 Architettura BERT

Tra i modelli basati sull'architettura Transformer, Bidirectional Encoder Representations from Transformers (BERT) [8] sfrutta un Transformer Encoder-only bidirezionale per produrre rappresentazioni contestuali dei token, risultando particolarmente adatto a compiti come il PoS tagging, nei quali la classificazione grammaticale dipende dal contesto frasale.

1.4.1 Tokenizzazione

L'input dell'encoder di BERT è costituito da token prodotti da WordPiece, un metodo di tokenizzazione che mantiene le parole intere oppure le suddivide in frammenti più piccoli, detti *subword*, selezionati da un vocabolario predefinito. Ad esempio, il tokenizer associato al modello `dbmdz/bert-base-italian-cased` rappresenta *bicicletta* con un singolo token, mentre suddivide *casetta* in `case` e `##tta` (il prefisso `##` indica la continuazione della parola). Questo approccio permette di rappresentare parole rare combinando frammenti già disponibili [8].

1.4.2 Struttura dell'encoder

Il modello `dbmdz/bert-base-italian-cased` adotta la configurazione BERT-Base, composta da 12 blocchi Transformer Encoder, 12 teste di attenzione per blocco e rappresentazioni vettoriali di dimensione 768 [12]. Le rappresentazioni vettoriali dei token (i.e. *token embeddings*) vengono sommate agli embedding posizionali e di segmento, che codificano rispettivamente la posizione nella sequenza e l'appartenenza ai segmenti di testo. Le rappresentazioni risultanti attraversano i 12 blocchi in successione [8]. La Figura 1.1 riassume il flusso delle rappresentazioni e la struttura interna di ciascun blocco.

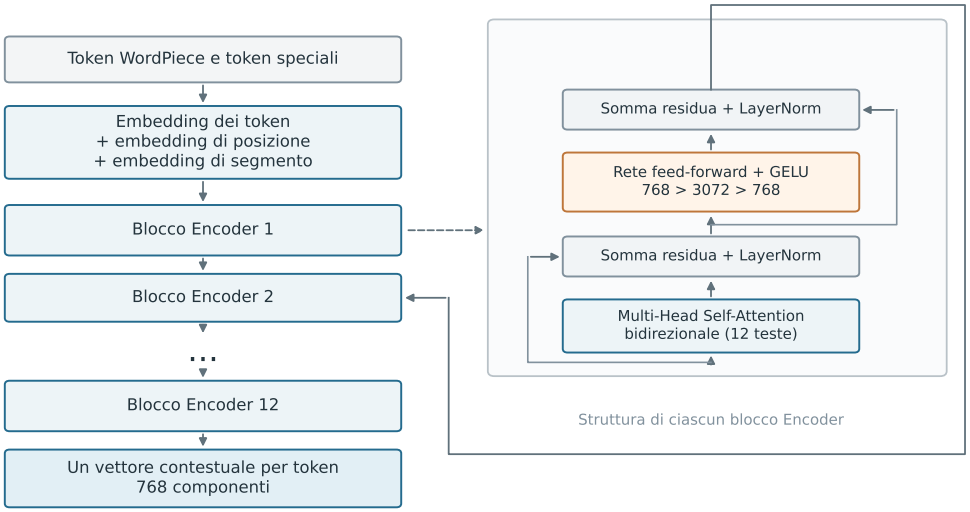


Figura 1.1: Struttura dell'encoder BERT-Base utilizzato nel progetto, con 12 teste di attenzione per blocco e vettori di 768 componenti. A sinistra, il flusso delle rappresentazioni procede dall'alto verso il basso attraverso i 12 blocchi; a destra, la struttura interna comune a tutti i blocchi, con attenzione bidirezionale, rete feed-forward e connessioni residue. La freccia di uscita mostra, come esempio, il passaggio dal primo al secondo blocco. Schema basato sull'architettura descritta da Devlin et al. [8].

Ogni blocco comprende una Multi-Head Self-Attention e una rete feed-forward. La prima esegue più operazioni di attenzione in parallelo, mediante proiezioni query, key e value apprese per ciascuna testa. Le uscite vengono concatenate e proiettate nello spazio di dimensione 768 [11]. Una particolarità di BERT è l'attenzione bidirezionale: ciascun token può utilizzare sia il contesto precedente sia quello successivo, senza vincoli causali [8].

Per ciascuna testa, l'attenzione è calcolata come:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{1.1}$$

dove Q, K e V sono matrici ottenute mediante proiezioni lineari apprese delle rappresentazioni in ingresso:

- **Query (Q):** rappresentano le informazioni ricercate dai token;
- **Key (K):** rappresentano le caratteristiche utilizzate per calcolare la compatibilità con le query;
- **Value (V):** contengono le informazioni da aggregare mediante i pesi di attenzione.

Il prodotto $QK^\top$ determina i punteggi di compatibilità, scalati per $\sqrt{d_k}$, dove $d_k = 768/12 = 64$ è la dimensionalità delle key per ciascuna testa. La softmax normalizza

tali punteggi in pesi di attenzione, utilizzati per calcolare una combinazione pesata dei vettori value.

La rete feed-forward applica indipendentemente a ciascun token due trasformazioni lineari, da 768 a 3 072 componenti e viceversa, separate da un'attivazione GELU. Entrambi i sottolivelli impiegano dropout, connessioni residue e Layer Normalization per favorire la regolarizzazione e la stabilità dell'addestramento [8, 11]. Al termine dei 12 blocchi, l'encoder restituisce un vettore contestuale di 768 componenti per ciascun token, utilizzabile nei compiti supervisionati.

1.4.3 Preaddestramento

Nel BERT originale il preaddestramento combina il Masked Language Modeling (MLM) e la Next Sentence Prediction (NSP). Nel MLM viene selezionato casualmente il 15% dei token della sequenza: di questi, l'80% viene sostituito con il token speciale `[MASK]`, il 10% con un token casuale e il restante 10% rimane invariato. Il modello deve predire i token originali nelle posizioni selezionate, sfruttando il contesto bidirezionale. La NSP richiede invece di determinare se due segmenti di testo siano consecutivi nel corpus, mediante una classificazione binaria [8].

Attraverso il preaddestramento, BERT apprende rappresentazioni linguistiche contestuali da grandi quantità di testo, che possono successivamente essere adattate a diversi compiti supervisionati mediante fine-tuning.

Il checkpoint adottato è stato preaddestrato su corpora italiani e, nella variante *cased*, mantiene la distinzione tra lettere maiuscole e minuscole durante la tokenizzazione [12].

1.5 Obiettivi del progetto

Il lavoro adatta e valuta BERT preaddestrato per il PoS tagging italiano, confrontandolo con sistemi di riferimento sul benchmark UD Italian-ISDT 2.15. Il compito principale consiste nella predizione dei tag UPOS, con gli split ufficiali, tokenizzazione e confini di frase gold. Il confronto diretto con UDPipe 2 sul medesimo test è affiancato dall'analisi dei risultati riportati in letteratura, considerando le differenze di release, condizioni di input e metriche che ne limitano la comparabilità [5, 13].

Gli esperimenti con encoder congelato e con Conditional Random Field (CRF) per la predizione della sequenza UPOS valutano rispettivamente il contributo del fine-tuning e della decodifica strutturata. La predizione congiunta dei tag UPOS e delle caratteristiche morfologiche del campo `FEATS` rappresenta un'estensione del compito principale. Le tre architetture vengono valutate su entrambi i compiti, per un totale di sei configurazioni. Il protocollo sperimentale e le metriche adottate sono descritti nel Capitolo 2.

Capitolo 2

Metodologia

2.1 Definizione dell'esperimento

L'esperimento confronta le tre architetture schematizzate nella Figura 2.1 e descritte nella Tabella 2.1, con encoder congelato o addestrabile e classificazione UPOS lineare o strutturata. Ogni architettura viene valutata prima sul solo compito UPOS e poi sul compito congiunto UPOS e FEATS, ottenendo sei configurazioni che permettono di osservare la variazione delle prestazioni UPOS durante la prosecuzione del training.

Architettura	Encoder	UPOS	FEATS (fase 2)	Learning rate
BERT congelato	Pesi fissi	Lineare	Lineari	10^{-3}
BERT fine-tuned	Addestrabile	Lineare	Lineari	$2 \cdot 10^{-5}$
BERT + CRF	Addestrabile	Lineare e CRF	Lineari	$2 \cdot 10^{-5}$

Tabella 2.1: Architetture e learning rate. Le teste presenti e l'eventuale CRF sono sempre addestrabili; le teste FEATS vengono aggiunte nella seconda fase.

Nella prima fase, a partire dal checkpoint BERT preaddestrato, l'ottimizzazione coinvolge la testa UPOS, l'eventuale CRF e, nelle architetture fine-tuned (i.e. con encoder addestrabile), l'encoder. Nella seconda fase, il checkpoint UPOS selezionato per la stessa architettura e lo stesso seed viene esteso aggiungendo le teste FEATS, inizializzate ex novo, mentre encoder, testa UPOS ed eventuale CRF mantengono i pesi appresi. L'addestramento prosegue su entrambi i compiti e, nelle architetture fine-tuned, le rispettive loss aggiornano l'encoder condiviso [14].

Ciascuna configurazione viene eseguita con i seed 42, 123 e 456, per un totale di 18 prove nel confronto finale, nove per fase, completando tutte le prove UPOS prima di avviare l'estensione a FEATS. L'impiego degli stessi seed permette di confrontare le configurazioni attraverso esecuzioni appaiate e di valutare la variabilità dei risultati mediante le statistiche descritte nella Sottosezione 2.7.3 [15].

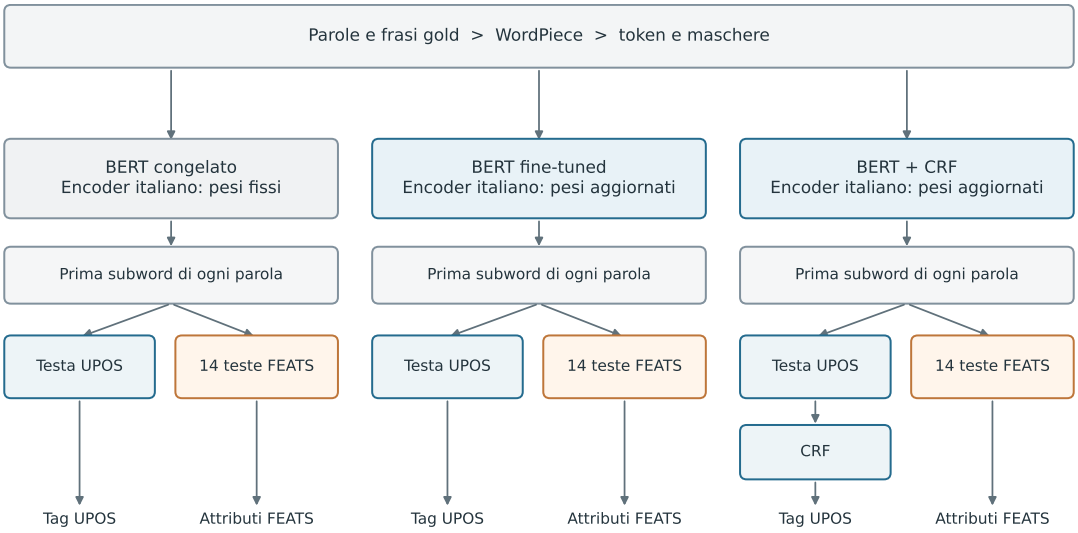


Figura 2.1: Schema delle tre architetture. Il colore grigio identifica l'encoder congelato; le teste sono sempre addestrabili. Le 14 teste `FEATS` vengono aggiunte nella seconda fase, mentre il CRF opera soltanto sulla sequenza UPOS.

2.2 Dati e annotazioni

2.2.1 Corpus e suddivisione

Si utilizza UD Italian-ISDT, release `r2.15`, scaricato dal repository ufficiale [5] e distribuito in file CoNLL-U (i.e. file testuali tabulari che riportano le parole delle frasi e le relative annotazioni linguistiche) [4]. Gli split ufficiali della Tabella 2.2 sono mantenuti, utilizzando il training per aggiornare i parametri e costruire gli inventari delle classi, il development set per selezionare il checkpoint e il test per la valutazione finale su dati esclusi dall'addestramento e dalla selezione.

Split	Frasi	Parole	Tag UPOS presenti
Train	13 121	276 014	17
Dev	564	11 907	17
Test	482	10 417	15

Tabella 2.2: Dimensioni degli split di UD Italian-ISDT 2.15; le parole corrispondono alle righe CoNLL-U con ID intero. Nel test sono assenti i tag `INTJ` e `PART`.

La lettura del corpus conserva le parole sintattiche identificate da un ID intero, escludendo le righe relative a token multiparola e nodi vuoti [4], mentre le forme delle parole e i confini di frase vengono acquisiti direttamente dalle annotazioni. Il protocollo assume pertanto segmentazione gold e confronta tutte le configurazioni sulle medesime parole, isolando la valutazione dell'etichettatura dalle operazioni di segmentazione. Un controllo delle sequenze di forme ha inoltre rilevato una frase identica tra training e sviluppo e nessuna tra training e test, mantenendo comunque la suddivisione ufficiale.

2.2.2 Rappresentazione di UPOS e FEATS

Gli inventari sono costruiti esclusivamente sul training e comprendono 17 tag UPOS e 14 attributi `FEATS`, per un totale di 40 coppie attributo-valore. Ogni parola riceve un'etichetta UPOS e un vettore morfologico con una componente per attributo, che assume uno dei valori osservati nel training oppure `<NONE>` quando l'attributo è assente.

Per esempio, `Gender=Masc|Number=Sing` viene rappresentato con `Masc` per Gender, `Sing` per Number e `<NONE>` per gli altri attributi; gli eventuali valori multipli sono ordinati e conservati come un'unica etichetta composta, secondo il formato CoNLL-U [4]. Le metriche mantengono anche le coppie gold assenti dal training, come `Typo=Yes` nel test, conteggiandole tra gli errori quando il modello non dispone della classe corrispondente.

2.3 Preprocessing e allineamento

Il tokenizer fast di `dbmdz/bert-base-italian-cased` viene applicato alle parole presegmentate, mantenendo la corrispondenza tra ciascuna subword e la parola di origine. Il vettore delle etichette gold (i.e. le annotazioni di riferimento fornite dal corpus) viene assegnato soltanto alla prima subword, escludendo token speciali, frammenti successivi e padding dalla loss e dalle metriche [16]; questa scelta mantiene una sola decisione supervisionata per parola, indipendentemente dal numero di frammenti prodotti dal tokenizer.

Nella variante CRF, gli score delle prime subword vengono raccolti in sequenze continue di parole e accompagnati da una maschera che identifica le posizioni valide, in modo che le transizioni siano apprese e decodificate tra parole consecutive. Il padding è dinamico per batch e tutte le frasi del corpus sono elaborate per intero; il preprocessing verifica che ogni parola sia coperta e interrompe l'esecuzione se tale condizione non è soddisfatta.

2.4 Modelli e funzione obiettivo

Le rappresentazioni dell'encoder alimentano una testa UPOS nella prima fase e 15 teste nella seconda, una per UPOS e 14 per `FEATS`, ciascuna con dropout di probabilità 0.1 seguito da una trasformazione lineare. Le teste condividono l'encoder ma hanno parametri distinti; quelle prive di CRF utilizzano cross-entropy mediata sulle parole valide durante l'addestramento e selezionano la classe con score massimo in predizione [16].

Nella variante CRF, gli score della testa UPOS costituiscono le emissioni di una catena lineare, combinate con parametri di transizione tra tag consecutivi per modellare le regolarità locali della sequenza. L'addestramento minimizza la log-verosimiglianza negativa della sequenza gold, normalizzata sul numero di parole del batch, mentre la decodifica Viterbi restituisce la sequenza con score complessivo massimo [17]. Il CRF opera soltanto su UPOS, lasciando lineari le teste `FEATS`.

Indicando con $\mathcal{A}$ l'insieme degli attributi morfologici, le funzioni obiettivo delle due fasi sono

$$\mathcal{L}^{(1)} = \mathcal{L}_{\text{UPOS}}, \qquad \mathcal{L}^{(2)} = \mathcal{L}_{\text{UPOS}} + \frac{\lambda}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathcal{L}_a, \qquad \lambda = 1. \tag{2.1}$$

La media delle loss FEATS mantiene il peso complessivo del compito morfologico indipendente dal numero delle teste, mentre la presenza della loss UPOS nella seconda fase consente di proseguire l'apprendimento del compito principale durante l'introduzione delle nuove supervisioni. Il coefficiente λ rimane fisso per tutte le architetture e tutti i seed, includendo nell'apprendimento anche le decisioni di assenza rappresentate da `<NONE>`.

2.5 Procedura di addestramento

Il notebook di implementazione, che raccoglie preparazione dei dati, addestramento e valutazione, è disponibile nel repository del progetto [18]. L'ottimizzazione utilizza AdamW [19] attraverso il `Trainer` di Transformers [16], con i learning rate della Tabella 2.1 e le impostazioni comuni della Tabella 2.3. Il valore più elevato della variante congelata riguarda soltanto le teste, mentre nelle architetture fine-tuned il learning rate viene applicato anche all'encoder; il seed viene impostato prima della creazione del modello e controlla l'inizializzazione delle nuove teste, l'ordine dei batch e le operazioni stocastiche.

Parametro	Valore
Epoche per fase	5
Batch di training / valutazione	8 / 16 frasi
Weight decay	0.01
Warmup	10% dei passi
Clipping della norma dei gradienti	1.0

Tabella 2.3: Impostazioni comuni alle due fasi; dopo il warmup, il learning rate decresce linearmente.

La seconda fase avvia una nuova ottimizzazione AdamW e un nuovo scheduler per altre cinque epoche. Le variazioni UPOS vanno quindi interpretate considerando l'effetto congiunto dell'estensione multitask, della prosecuzione dell'addestramento e del criterio di selezione.

L'esecuzione su NVIDIA GeForce RTX 4070 SUPER, con 12 GB di memoria, utilizza precisione mista FP16 per accelerare il training e ridurre l'occupazione di memoria.

2.6 Selezione del checkpoint

Al termine di ogni epoca il modello viene valutato sul development set e, nella prima fase, viene selezionato il checkpoint che massimizza la macro-F1 UPOS sui 17 tag

del training. Nella seconda fase si utilizza invece il punteggio $S_{\text{dev}} = (F_{\text{U}} + F_{\text{F}})/2$, dove F_{U} indica la stessa macro-F1 UPOS e F_{F} la macro-F1 FEATS sulle coppie presenti nel development, escludendo `<NONE>`. La media assegna uguale peso ai due compiti nella selezione. Nella seconda fase viene salvato un checkpoint al termine di ciascuna delle cinque epoche; per la valutazione finale si utilizza quello con il punteggio migliore sul development set.

La Figura 2.2 mostra che la loss di training continua a diminuire, mentre quella sul development si stabilizza: gli aggiornamenti successivi migliorano quindi soprattutto l'adattamento ai dati di addestramento. Nella fase congiunta, la macro-F1 UPOS sul development diminuisce mentre cresce quella `FEATS`; il punteggio medio dei due compiti aumenta perché il miglioramento morfologico compensa la riduzione su UPOS, spiegando la selezione di checkpoint nelle ultime epoche.

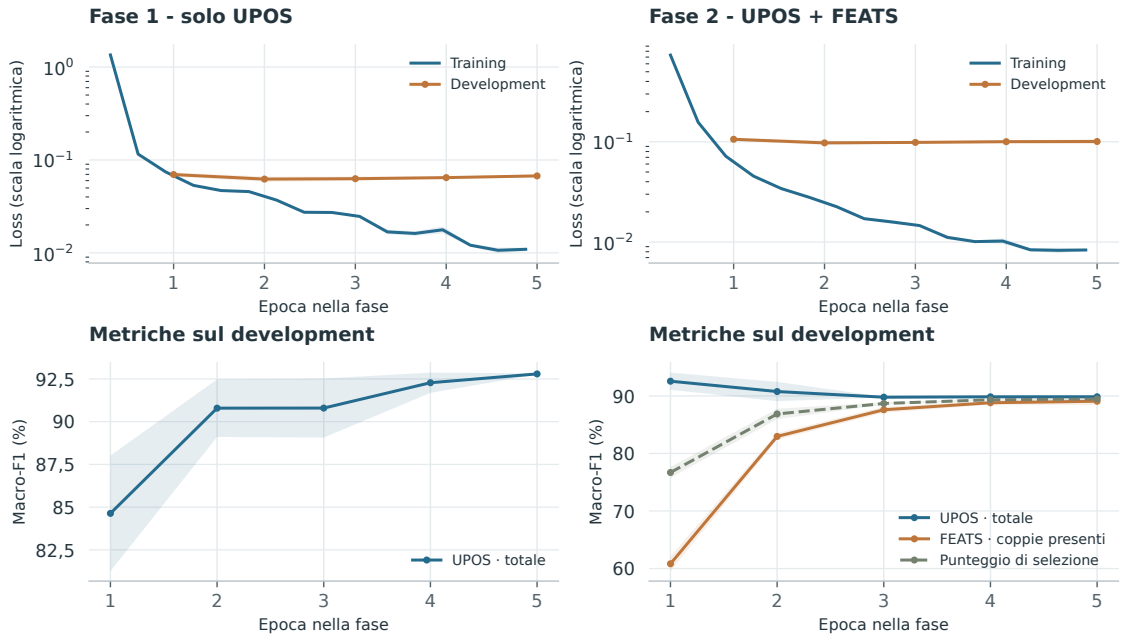


Figura 2.2: Andamento di BERT fine-tuned durante le due fasi: loss di training e sul development in alto, macro-F1 sul development in basso. Le linee indicano le medie sui tre seed e le aree ombreggiate una deviazione standard campionaria.

2.7 Valutazione

Le predizioni BERT vengono ricondotte alle parole originali utilizzando quelle associate alla prima subword di ciascuna parola. Si valutano UPOS per tutte le configurazioni e, nella seconda fase, anche FEATS e accuracy congiunta. Le stesse metriche sono applicate a UDPipe 2, verificando ordine ed etichette gold.

2.7.1 Metriche UPOS

L'accuracy misura la frazione di parole con tag corretto e descrive la prestazione complessiva, alla quale viene affiancata la macro-F1 per attribuire lo stesso peso alle categorie, comprese quelle meno frequenti. Per ciascun tag k si conteggiano veri positivi (TP_k), falsi positivi (FP_k) e falsi negativi (FN_k), dai quali si ricava l'F1 come media armonica di precision e recall [20]:

$$F_{1,k} = \frac{2TP_k}{2TP_k + FP_k + FN_k}, \qquad F_1^{\text{macro}} = \frac{1}{|C|} \sum_{k \in C} F_{1,k}. \tag{2.2}$$

La versione *totale* include i 17 tag del training, mentre quella sulle classi *presenti* considera soltanto i tag con supporto gold positivo nello split valutato, che nel test sono 15 per l'assenza di `INTJ` e `PART`. Ponendo a zero gli F1 con denominatore nullo, la macro-F1 totale sul test ha quindi un massimo teorico di $15/17 \simeq 88{,}24\%$ anche con tutte le predizioni corrette, mentre quella sulle classi presenti mantiene come massimo il 100%.

Considerando tutti i tag, precision, recall e F1 con media *micro* coincidono con l'accuracy, poiché ogni parola riceve una sola etichetta [20]; il riepilogo utilizza pertanto accuracy e macro-F1, che sintetizzano correttezza complessiva e prestazioni per categoria. Precision e recall delle singole classi restano disponibili per l'analisi degli errori, distinguendo la correttezza delle assegnazioni dalla copertura delle occorrenze gold.

2.7.2 Metriche FEATS

L'accuracy FEATS principale richiede che l'intero insieme di coppie attributo-valore di una parola coincida con quello gold, valutando congiuntamente la presenza degli attributi e la correttezza dei loro valori. Indicando con G_i e P_i gli insiemi gold e predetto per la parola i, si calcola

$$\text{Acc}_{\text{FEATS}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[G_i = P_i], \tag{2.3}$$

includendo anche gli insiemi vuoti, mentre una misura aggiuntiva conta le decisioni corrette sulle coppie parola-attributo e comprende la classe `<NONE>`. Quest'ultima misura viene interpretata insieme all'accuracy dell'insieme completo, poiché la frequenza degli attributi assenti può incidere sensibilmente sul suo valore; gli attributi valutati sono l'unione di quelli di training, gold e predetti.

La macro-F1 tratta ogni coppia attributo-valore (e.g. `Gender=Fem`) come una classe e ne conteggia veri positivi, falsi positivi e falsi negativi sulla presenza nelle singole parole, escludendo `<NONE>` dalla media. La versione *totale* comprende le 40 coppie del training e le ulteriori coppie osservate nelle annotazioni gold o nelle predizioni, mentre quella sulle classi *presenti* considera soltanto le coppie con supporto gold positivo, mantenendo nel conteggio degli errori anche le annotazioni sconosciute al modello.

Si riporta inoltre l'accuracy congiunta, definita come la frazione di parole per le quali risultano corretti sia UPOS sia l'intero insieme FEATS, affiancando alle misure complessive le metriche dei singoli attributi per esaminarne le differenze di comportamento.

2.7.3 Aggregazione e analisi degli errori

Per ogni configurazione si riportano media e deviazione standard campionaria sui tre seed [15], conservando le misure individuali. Le variazioni UPOS tra le fasi sono appaiate per architettura e seed ed espresse in punti percentuali. L'analisi degli errori considera confusioni UPOS, falsi positivi e negativi delle coppie `FEATS` e metriche per classe.

2.8 Confronto con UDPipe 2

UDPipe 2 `italian-isdt-ud-2.15-241121` viene valutato sullo stesso test con parole e confini gold, rimuovendo le annotazioni grammaticali [13]. Si verificano modello, hash del test e allineamento, ricalcolando le metriche UPOS e FEATS con le convenzioni di BERT.

Con parole allineate, l'F1 UPOS coincide con l'accuracy. Le metriche FEATS comprendono tutti gli attributi, inclusi quelli specifici del treebank e assenti dal training, mentre il valutatore UD limita *UFeats* agli attributi universali e include XPOS in *AllTags*. UDPipe impiega anche BERT multilingue: il confronto riguarda quindi sistemi completi [13].

Incertezza del confronto appaiato. Il bootstrap appaiato per frase [21] stima l'incertezza degli scarti BERT meno UDPipe: per ciascuna di 10 000 repliche si estraggono con reinserimento 482 frasi, usando lo stesso campione per tutti i sistemi. Le metriche sono ricalcolate dai conteggi complessivi, mantenendo le 15 classi UPOS del test e F1 nullo ai denominatori nulli; gli intervalli percentile al 95% sono separati per fase e seed.

Gli intervalli descrivono l'incertezza sul test per modelli già addestrati, senza includere la variabilità del training. Il ricampionamento preserva le parole di ogni frase, assume indipendenza tra frasi e risente delle categorie rare; codice e parametri sono nel repository [18].

2.9 Riproducibilità

Per ogni prova si conservano modello selezionato, cronologia, predizioni e metriche, insieme a revisioni, hash dei dati, inventari, seed, iperparametri e versioni software. Si registra inoltre l'origine dei pesi UPOS; codice e istruzioni, compresi gli script di confronto dagli output salvati, sono nel repository del progetto [18].

Capitolo 3

Risultati

La valutazione del modello fine-tuned per UPOS è incentrata sul confronto con UDPipe 2, riprodotto nelle stesse condizioni sul benchmark UD Italian-ISDT 2.15, e sul posizionamento rispetto ai risultati pubblicati. Il test comprende 10 417 parole; per le configurazioni BERT si riportano media e deviazione standard campionaria sui tre seed delle 18 prove finali. I confronti tra architetture e l'estensione `FEATS` completano l'analisi del sistema.

3.1 Confronto UPOS con UDPipe 2

BERT fine-tuned sul solo compito UPOS raggiunge un'accuracy del $98{,}81 \pm 0{,}09\%$, rispetto al 98,75% di UDPipe 2. Dopo l'estensione congiunta a UPOS e `FEATS`, la stessa architettura raggiunge il $98{,}84 \pm 0{,}05\%$. La Figura 3.1 evidenzia il confronto sulle due metriche principali; il confronto utilizza le stesse parole e gli stessi confini di frase gold, con metriche ricalcolate dalle predizioni allineate utilizzando le medesime convenzioni per tutti i sistemi, secondo il protocollo della Sezione 2.8.

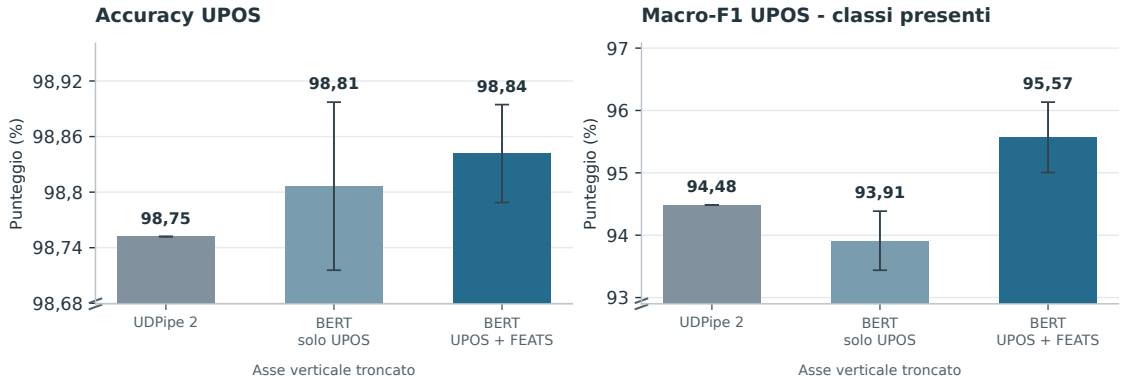


Figura 3.1: Accuracy e macro-F1 UPOS sulle classi presenti per UDPipe 2 e BERT fine-tuned, prima e dopo la fase congiunta con `FEATS`. Per BERT le colonne mostrano la media sui tre seed e le barre di errore la deviazione standard; UDPipe corrisponde a una sola esecuzione. Gli assi verticali sono troncati per evidenziare gli scarti.

Il confronto evidenzia accuracy vicine al riferimento e un miglioramento della macro-F1 dopo l'estensione morfologica. La Tabella 3.1 completa questa lettura con i risultati delle tre architetture in entrambe le fasi.

Architettura	Accuracy	Macro-F1 presenti	Macro-F1 totale
UDPipe 2 (UD 2.15)	98,75	94,48	83,37
Fase 1: UPOS			
BERT congelato	$96{,}36 \pm 0{,}07$	$89{,}61 \pm 0{,}26$	$79{,}07 \pm 0{,}23$
BERT fine-tuned	$98{,}81 \pm 0{,}09$	$93{,}91 \pm 0{,}47$	$82{,}86 \pm 0{,}42$
BERT + CRF	$98{,}79 \pm 0{,}06$	$94{,}63 \pm 0{,}43$	$83{,}50 \pm 0{,}38$
Fase 2: UPOS + `FEATS`			
BERT congelato	$96{,}60 \pm 0{,}05$	$89{,}94 \pm 0{,}26$	$79{,}36 \pm 0{,}23$
BERT fine-tuned	$98{,}84 \pm 0{,}05$	$95{,}57 \pm 0{,}56$	$84{,}33 \pm 0{,}50$
BERT + CRF	$98{,}82 \pm 0{,}05$	$95{,}49 \pm 0{,}64$	$84{,}25 \pm 0{,}57$

Tabella 3.1: Risultati UPOS sul medesimo test UD Italian-ISDT 2.15 con segmentazione gold, espressi in percentuale come media ± deviazione standard. Per UDPipe si riporta un'unica esecuzione; la macro-F1 sulle classi presenti utilizza 15 tag, quella totale tutti i 17 tag.

Il margine medio di accuracy rispetto a UDPipe è di circa 0,06 punti percentuali nella prima fase e 0,09 nella seconda. UDPipe commette 130 errori, mentre BERT fine-tuned della seconda fase ne commette 127, 118 e 117 nei tre seed. La differenza corrisponde quindi a poche parole su oltre diecimila e viene esaminata anche attraverso gli intervalli del confronto appaiato.

La macro-F1 sulle 15 classi presenti rende più articolato il confronto: nella prima fase BERT fine-tuned ottiene il 93,91%, contro il 94,48% di UDPipe, pur avendo un'accuracy media leggermente maggiore. Dopo la fase congiunta raggiunge il 95,57%, con uno scarto medio di 1,09 punti rispetto al riferimento. Circa 0,93 punti di questo scarto provengono dalla categoria `X`, presente soltanto 13 volte, il cui F1 passa dal 37,50% di UDPipe al 51,48% medio di BERT. Il vantaggio macro richiede quindi cautela rispetto alle classi rare. La macro-F1 sui 17 tag segue lo stesso andamento, con la differenza di scala dovuta alle due classi assenti dal test descritta nella Sottosezione 2.7.1.

La Tabella 3.2 quantifica l'incertezza degli scarti di BERT fine-tuned rispetto a UDPipe, usando il ricampionamento per frase descritto nella Sezione 2.8. Gli intervalli di accuracy e macro-F1 comprendono lo zero per tutti i seed in entrambe le fasi: sul test disponibile, quindi, la direzione dei piccoli scarti osservati non risulta stabile rispetto al ricampionamento. Questo esito non dimostra equivalenza tra i sistemi; gli intervalli più ampi della macro-F1 riflettono anche il peso delle categorie con pochi esempi.

Seed	Δ accuracy [IC 95%]	Δ macro-F1 presenti [IC 95%]
Fase 1: UPOS		
42	−0,05 [−0,22; +0,12]	−0,95 [−4,20; +1,35]
123	+0,12 [−0,06; +0,31]	−0,04 [−1,88; +1,60]
456	+0,09 [−0,08; +0,26]	−0,72 [−2,01; +0,37]
Fase 2: UPOS + `FEATS`		
42	+0,03 [−0,15; +0,22]	+0,95 [−1,81; +3,86]
123	+0,12 [−0,08; +0,31]	+0,60 [−3,35; +4,55]
456	+0,12 [−0,06; +0,32]	+1,70 [−0,28; +4,66]

Tabella 3.2: Differenze UPOS di BERT fine-tuned rispetto a UDPipe, in punti percentuali, e intervalli bootstrap appaiati al 95% da 10 000 ricampionamenti di frasi. Ogni seed viene valutato separatamente; gli intervalli sono marginali, senza correzione per confronti multipli.

3.2 Posizionamento rispetto alla letteratura

Il confronto con la letteratura considera sistemi pertinenti al PoS tagging italiano, con particolare riferimento ai risultati su Italian-ISDT. La Tabella 3.3 raccoglie sia encoder Transformer italiani sia modelli generativi adattati all'analisi sintattica, riportando per ciascuno la metrica e le condizioni di valutazione disponibili.

Sistema e fonte	UPOS (%)	Metrica e condizioni dichiarate
UmBERTo [22]	98,78 ± 0,08	Accuracy; UD-ISDT 2.5; almeno 5 esecuzioni.
BERT italiano *cased* [23]	98,84 ± 0,06	Accuracy; ISDT, commit `f20fa2b`; 5 esecuzioni.
BERT italiano XXL *cased* [23]	98,90 ± 0,03	Accuracy; stesso protocollo di BERT italiano.
UmBERTo Commoncrawl *cased* [24]	98,977	Accuracy; Italian-ISDT, release non specificata.
Gemma-2-9B multilingue [25]	98,8	F1 UPOS; UD 2.15; output generati.
LLM con LoRA, quattro modelli [26]	98,3–98,8	F1 UPOS; UD 2.15; LoRA, generazione in due passi.

Tabella 3.3: Riferimenti di letteratura per UPOS su Italian-ISDT. I valori sono quelli delle fonti e non costituiscono una graduatoria su un protocollo comune; i risultati dei repository degli autori sono distinti da quelli degli articoli.

Le accuracy del modello sviluppato, pari al 98,81% sul solo UPOS e al 98,84% dopo l'estensione a `FEATS`, sono in accordo con i risultati riportati per i Transformer italiani. Tamburini [22] ottiene il 98,78% con UmBERTo su UD-ISDT 2.5; il repository di Schweter [23] riporta il 98,84% per BERT italiano *cased*, utilizzato anche in questo progetto, e il 98,90% per la variante XXL. Il repository ufficiale di UmBERTo [24] documenta un valore più alto, pari al 98,977%.

Le differenze architetturali riguardano anche il modo in cui viene utilizzata l'informazione morfologica. Nel precedente sistema di Tamburini [27], valutato sui benchmark EVALITA, una rete BiLSTM-CRF riceve in ingresso rappresentazioni lessicali arricchite con le analisi prodotte da AnIta, un analizzatore morfologico per l'italiano. Nel presente progetto, invece, la morfologia viene appresa come compito supervisionato aggiuntivo mediante le teste `FEATS`, senza fornire al modello le analisi di uno strumento esterno.

Le condizioni sui dati sono riportate per contestualizzare i valori della tabella. Il confronto dei file ISDT del commit `f20fa2b`, usato da Schweter, con UD 2.15 conferma che il test conserva le stesse 482 frasi e 10 417 parole, con forme e confini di frase identici, ma contiene cinque etichette UPOS differenti; anche training e development presentano differenze, come verificabile con lo script di confronto incluso nel repository del progetto [18]. Per il risultato del repository UmBERTo, la tabella originale non specifica la release del corpus e i dettagli del protocollo.

I lavori di Matsuda et al. [25] e Matsuda e Asahara [26] ampliano il confronto ai modelli generativi su UD 2.15. Nel primo, Gemma-2-9B multilingue raggiunge 98,8 nella metrica UPOS della valutazione UD. Nel secondo, gli LLM sono adattati mediante LoRA e producono annotazioni con una procedura denominata `2-step-no-form`: prima generano i tag UPOS, poi le teste e le relazioni sintattiche, senza ripetere le forme delle parole nell'output. Su Italian-ISDT gli autori riportano F1 UPOS del 98,7% per Gemma-2-9B, 98,8% per Llama-3.1-8B e Qwen2.5-7B e 98,3% per OLMo-2-7B.

Nel complesso, il modello sviluppato raggiunge prestazioni in linea con la letteratura sul PoS tagging italiano. Il confronto comprende approcci diversi per preaddestramento, architettura e impiego dell'informazione morfologica, mostrando come il fine-tuning di BERT italiano costituisca una soluzione efficace per il compito considerato.

3.3 Confronti interni tra le configurazioni BERT

I confronti interni mostrano il contributo del fine-tuning: nella prima fase l'accuracy passa dal 96,36% dell'encoder congelato al 98,81% di BERT fine-tuned. L'aggiunta del CRF mantiene un'accuracy simile e migliora la macro-F1 sulle classi presenti, con un beneficio legato soprattutto alla categoria rara `X`, discussa nella Sezione 3.5. Il ciclo di addestramento richiede in media 6,75 minuti per BERT fine-tuned e 10,71 minuti con CRF, sulla stessa GPU e includendo valutazioni e salvataggi.

La Figura 3.2 riassume il passaggio alla seconda fase: l'incremento medio di accuracy rimane contenuto, mentre la macro-F1 sulle classi presenti mostra un miglioramento più evidente, soprattutto per BERT fine-tuned.

3.4 Estensione alla predizione `FEATS`

La predizione delle caratteristiche morfologiche estende il compito principale UPOS. La Figura 3.3 mette a confronto BERT fine-tuned e UDPipe 2 sulle due metriche

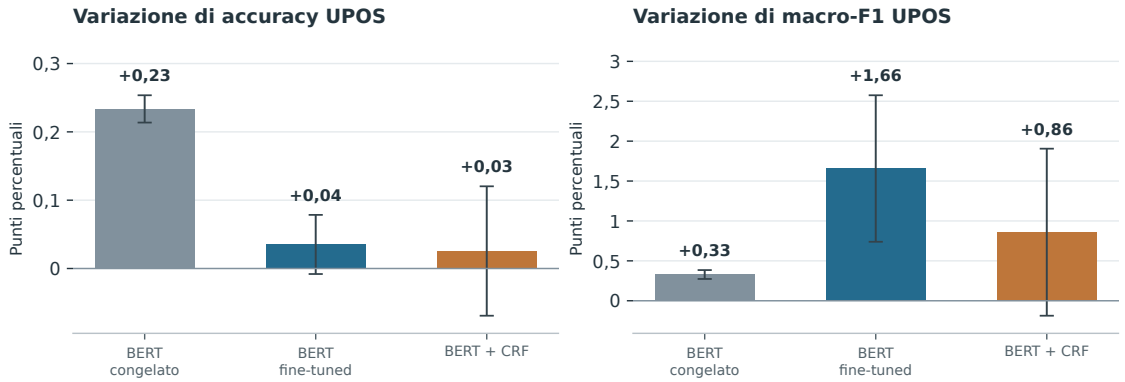


Figura 3.2: Variazioni UPOS tra la fase congiunta e quella sul solo UPOS, calcolate appaiando i risultati dello stesso seed. Le colonne indicano la variazione media e le barre di errore la deviazione standard campionaria sui tre seed. La macro-F1 considera le classi presenti; le scale dei due pannelli sono differenti.

morfologiche principali; la Tabella 3.4 completa il quadro con tutte le configurazioni della seconda fase sul medesimo test con segmentazione gold. L'accuracy esatta richiede la corrispondenza dell'intero insieme `FEATS` di una parola; le macro-F1 attribuiscono uguale peso alle coppie attributo-valore. Per il riferimento si riutilizzano le predizioni già disponibili, valutate con le convenzioni della Sezione 2.8.

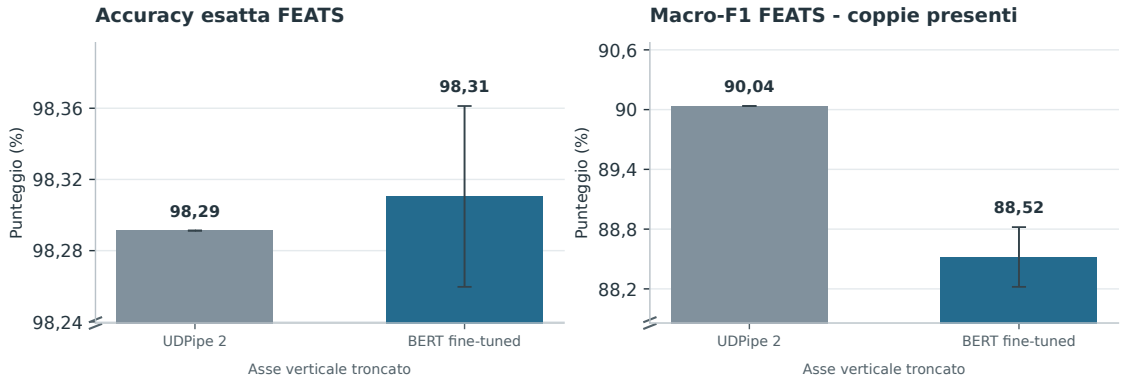


Figura 3.3: Accuracy esatta e macro-F1 sulle coppie `FEATS` presenti: confronto tra BERT fine-tuned e UDPipe 2. Per BERT si riportano media e deviazione standard sui tre seed; per UDPipe una sola esecuzione. Gli assi verticali sono troncati.

Come mostra la Figura 3.3, BERT fine-tuned e UDPipe raggiungono un'accuracy esatta `FEATS` molto vicina, mentre la macro-F1 sulle coppie presenti favorisce UDPipe. Quasi tutto il divario macro dipende da una coppia con tre occorrenze, come approfondito nella Sezione 3.5; questo risultato descrive anche la sensibilità della metrica alle classi rare.

Nei confronti interni, l'accuracy `FEATS` passa dall'87,68% del modello congelato al 98,31% di BERT fine-tuned e la macro-F1 sulle coppie presenti dall'82,45% all'88,52%. Gli scarti tra i due modelli fine-tuned rimangono contenuti: la differenza media di accuracy `FEATS` è di 0,06 punti e il CRF risulta lievemente migliore in due seed

Sistema	Accuracy FEATS	Macro-F1 presenti	Accuracy congiunta	Macro-F1 totale
UDPipe 2	98,29	90,04	97,94	83,45
BERT congelato	$87{,}68 \pm 0{,}05$	$82{,}45 \pm 0{,}27$	$86{,}48 \pm 0{,}07$	$76{,}42 \pm 0{,}25$
BERT fine-tuned	$98{,}31 \pm 0{,}05$	$88{,}52 \pm 0{,}30$	$97{,}91 \pm 0{,}09$	$82{,}04 \pm 0{,}28$
BERT + CRF	$98{,}25 \pm 0{,}09$	$88{,}37 \pm 0{,}23$	$97{,}82 \pm 0{,}13$	$81{,}90 \pm 0{,}21$

Tabella 3.4: Risultati morfologici sul test UD Italian-ISDT 2.15 con segmentazione gold. Per BERT si riportano media ± deviazione standard sui tre seed; per UDPipe un'unica esecuzione. Tutti i valori sono percentuali e ricalcolati con le stesse convenzioni. Le macro-F1 considerano 41 coppie attributo-valore complessive e 38 presenti nel test.

su tre, pertanto l'ordinamento delle medie richiede cautela. Inoltre, le teste `FEATS` sono lineari in entrambe le architetture, mentre il CRF agisce soltanto su UPOS; le differenze morfologiche riflettono quindi l'apprendimento congiunto delle teste e dell'encoder condiviso, oltre alla selezione del checkpoint.

L'accuracy sulle singole decisioni parola-attributo raggiunge il 99,83%, ma comprende numerose decisioni di assenza. Anche l'accuracy esatta include le 4 055 parole con insieme gold vuoto, mentre la macro-F1 attribuisce uguale peso alle coppie frequenti e rare: queste differenze spiegano la distanza tra le misure.

La selezione congiunta individua checkpoint delle epoche 4–5 per BERT fine-tuned e 3–5 per BERT + CRF, mentre il modello congelato seleziona sempre la quinta epoca; la deviazione standard dell'accuracy `FEATS` rimane entro 0,10 punti in tutte le architetture.

3.5 Analisi degli errori

Le principali confusioni UPOS coinvolgono nomi comuni, nomi propri e aggettivi: sommando le tre esecuzioni di BERT fine-tuned della prima fase, si osservano 46 assegnazioni da `NOUN` a `PROPN`, 38 nella direzione opposta e 29 da `NOUN` a `ADJ`. La Figura 3.4 mostra che questi scambi restano prevalenti dopo l'estensione morfologica.

La categoria `X`, presente soltanto 13 volte, chiarisce la sensibilità della macro-F1: nella prima fase il suo F1 medio passa dal 26,36% con classificazione lineare al 37,37% con CRF, spiegando quasi interamente il vantaggio macro della seconda architettura. Dopo la fase congiunta, i valori raggiungono rispettivamente il 51,48% e il 53,08%, contribuendo agli incrementi della media per categoria a fronte di variazioni ridotte dell'accuracy.

Per `FEATS`, gli errori in valore assoluto si concentrano soprattutto su numero e genere, che hanno elevato supporto: e.g., `Number=Sing` accumula 119 falsi positivi e 108 falsi negativi nelle tre prove di BERT fine-tuned, pur mantenendo un F1 medio del 99,07%. Restano più difficili alcune coppie rare, tra cui `Foreign=Yes`, con 11 occorrenze e F1 medio del 29,30%, mentre `Degree=Abs` e `PronType=Exc`, presenti rispettivamente tre volte e una, ricevono F1 nullo in tutte le configurazioni

BERT. Anche l'unica occorrenza di `Typo=Yes` resta errata, poiché la coppia è assente dall'inventario di training.

UDPipe riconosce due delle tre occorrenze di `Degree=Abs`, con un F1 del 57,14%. Questa coppia spiega circa 1,50 dei 1,52 punti di vantaggio macro su BERT fine-tuned, mentre le differenze sulle altre coppie quasi si compensano.

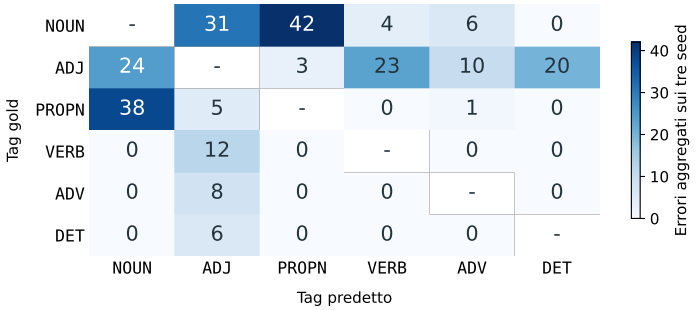


Figura 3.4: Confusioni UPOS di BERT fine-tuned: righe gold, colonne predette. Ogni cella conta gli errori aggregati sui tre seed; il blu più scuro indica più errori, ad esempio 42 da `NOUN` a `PROPN`. La diagonale delle predizioni corrette è esclusa.

3.5.1 Esempi contestuali

La Tabella 3.5 raccoglie quattro casi illustrativi, con predizioni BERT identiche nei tre seed. Comprendono successi ed errori di entrambi i sistemi e un'omissione morfologica, senza costituire un campione rappresentativo.

Estratto	Campo	Gold	UDPipe	BERT
Il Presidente **eletto** nomina e revoca i componenti della Giunta.	UPOS	`ADJ`	`VERB`	`ADJ`
Chi è il **Ministro** degli Esteri italiano?	UPOS	`PROPN`	`PROPN`	`NOUN`
... l'operazione è considerata già **conclusa**.	UPOS	`ADJ`	`VERB`	`VERB`
Recentissima anche la creazione di "Splatter" ...	`Degree`	`Abs`	`Abs`	`<NONE>`

Tabella 3.5: Esempi sul test: parola bersaglio in grassetto ed estratti abbreviati con puntini. L'ultima riga confronta `Degree`; `<NONE>` ne indica l'assenza.

Nel gold, *eletto* identifica *Presidente* come aggettivo: BERT riproduce questa annotazione, mentre UDPipe assegna `VERB` e caratteristiche di participio. Anche *conclusa* è annotata come `ADJ`, ma entrambi i sistemi la trattano come verbo; i due casi mostrano come la stessa origine participiale possa accompagnarsi a decisioni diverse nel contesto.

Per *Ministro*, UDPipe segue il gold `PROPN`, mentre BERT assegna `NOUN`: il caso richiama il trattamento dei titoli istituzionali nel treebank, rispetto al quale viene calcolata la correttezza. In *Recentissima*, invece, entrambi riconoscono `ADJ`, ma BERT omette `Degree=Abs` pur predicendo correttamente genere e numero; distinguere la categoria grammaticale dall'insieme morfologico permette così di localizzare l'errore.

Capitolo 4

Conclusioni

Il progetto ha adattato BERT italiano al PoS tagging sul benchmark UD Italian-ISDT 2.15, utilizzando gli split ufficiali e la segmentazione gold. Il modello fine-tuned raggiunge un'accuracy UPOS di circa il 98,81% nella prima fase e, dopo l'estensione congiunta a UPOS e `FEATS`, ottiene un'accuracy UPOS di circa il 98,84% e una macro-F1 sulle classi presenti di circa il 95,57%, rispetto al 98,75% e al 94,48% di UDPipe 2. Le prestazioni sono in accordo con la letteratura sul PoS tagging italiano, mentre gli intervalli appaiati sul test comprendono lo zero: i risultati sostengono l'efficacia del modello nelle condizioni considerate, con incertezza sui piccoli scarti rispetto a UDPipe.

Per la morfologia, BERT fine-tuned raggiunge il 98,31% di accuracy esatta `FEATS`, vicino al 98,29% di UDPipe, mentre la macro-F1 sulle coppie presenti favorisce il riferimento, con il 90,04% contro l'88,52% di BERT. Le due metriche restituiscono quindi un quadro complementare: alla buona correttezza complessiva si affiancano difficoltà sulle caratteristiche rare, che incidono maggiormente sulla macro-F1.

I confronti interni mostrano che il fine-tuning migliora nettamente le prestazioni rispetto all'encoder congelato, mentre il CRF offre benefici più circoscritti, soprattutto sulle categorie rare, a fronte di tempi di addestramento maggiori. L'estensione alla morfologia si accompagna a un miglioramento delle prestazioni UPOS, in particolare della macro-F1 sulle classi presenti.

Un possibile sviluppo consiste nell'aggiungere un CRF a ciascuna testa `FEATS`, estendendo la decodifica strutturata alle sequenze dei singoli attributi; i benefici contenuti osservati su UPOS suggeriscono che anche questa estensione potrebbe produrre guadagni limitati. Miglioramenti più consistenti potrebbero richiedere un'architettura capace di modellare esplicitamente le dipendenze tra categorie grammaticali e attributi morfologici, rappresentando anche i vincoli di compatibilità tra le etichette, oltre alle relazioni tra parole consecutive.

La valutazione su un solo treebank, con segmentazione gold e tre seed, circoscrive le conclusioni, suggerendo ulteriori analisi statistiche appaiate e valutazioni su altri treebank o da testo grezzo.

Bibliografia

[1] Christopher D. Manning. Part-of-speech tagging from 97% to 100%: Is it time for some linguistics? In *Computational Linguistics and Intelligent Text Processing*, volume 6608 of *Lecture Notes in Computer Science*, pages 171–189. Springer, 2011. doi:10.1007/978-3-642-19400-9_14.

[2] Universal Dependencies contributors. Universal dependencies: Framework and documentation. Sito ufficiale del progetto Universal Dependencies. URL: https://universaldependencies.org/.

[3] Universal Dependencies contributors. Universal POS tags. Universal Dependencies v2 – Linee guida per l'annotazione delle categorie grammaticali. URL: https://universaldependencies.org/u/pos/.

[4] Universal Dependencies contributors. CoNLL-U format. Specifiche di Universal Dependencies v2. URL: https://universaldependencies.org/format.html.

[5] Universal Dependencies contributors. UD Italian-ISDT. Treebank, release r2.15, 2024. URL: https://github.com/UniversalDependencies/UD_Italian-ISDT/tree/r2.15.

[6] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi:10.1109/TKDE.2008.239.

[7] Barbara Plank, Anders Søgaard, and Yoav Goldberg. Multilingual part-of-speech tagging with bidirectional long short-term memory models and auxiliary loss. In *Proceedings of ACL 2016 (Volume 2: Short Papers)*, pages 412–418. Association for Computational Linguistics, 2016. doi:10.18653/v1/P16-2067.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186. Association for Computational Linguistics, 2019. doi:10.18653/v1/N19-1423.

[9] Yoav Goldberg. A primer on neural network models for natural language processing. *Journal of Artificial Intelligence Research*, 57:345–420, 2016. doi:10.1613/jair.4992.

[10] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL: https://papers.nips.cc/paper_files/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[12] DBMDZ. BERT base italian cased. Model card, Hugging Face. Consultato il 7 novembre 2026. URL: https://huggingface.co/dbmdz/bert-base-italian-cased.

[13] ÚFAL, Charles University. UDPipe 2 models: Universal Dependencies 2.15. Documentazione dei modelli. Consultato il 7 novembre 2026. URL: https://ufal.mff.cuni.cz/udpipe/2/models.

[14] Dan Kondratyuk and Milan Straka. 75 languages, 1 model: Parsing Universal Dependencies universally. In *Proceedings of EMNLP-IJCNLP 2019*, pages 2779–2795. Association for Computational Linguistics, 2019. doi:10.18653/v1/D19-1279.

[15] Nils Reimers and Iryna Gurevych. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of EMNLP 2017*, pages 338–348. Association for Computational Linguistics, 2017. doi:10.18653/v1/D17-1035.

[16] Hugging Face. Token classification. Documentazione di Transformers, versione 4.46.3. URL: https://huggingface.co/docs/transformers/v4.46.3/tasks/token_classification.

[17] John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of ICML 2001*, pages 282–289. Morgan Kaufmann, 2001. URL: https://repository.upenn.edu/cis_papers/159.

[18] Pierre Errico. Pos tagging italiano con bert: codice e materiali del progetto. Repository GitHub, 2026. URL: https://github.com/USERNAME/REPOSITORY.

[19] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL: https://arxiv.org/abs/1711.05101.

[20] scikit-learn developers. Metrics and scoring: Quantifying the quality of predictions. Documentazione di scikit-learn, versione 1.8. Consultato l'8 novembre 2026. URL: https://scikit-learn.org/1.8/modules/model_evaluation.html.

[21] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, 2018. URL: https://aclanthology.org/P18-1128/, doi:10.18653/v1/P18-1128.

[22] Fabio Tamburini. How BERTology changed the state-of-the-art also for italian NLP. In *Proceedings of CLiC-it 2020*, pages 313–319. CEUR Workshop Proceedings, 2020. URL: https://aclanthology.org/2020.clicit-1.48/.

[23] Stefan Schweter. Italian BERT and ELECTRA models. Repository degli autori, 2020. Test ISDT al commit f20fa2b. URL: https://github.com/stefan-it/italian-bertelectra.

[24] Loreto Parisi, Simone Francia, and Paolo Magnani. UmBERTo: an italian language model trained with whole word masking. Repository degli autori, 2020. Consultato il 9 novembre 2026. URL: https://github.com/musixmatchresearch/umberto.

[25] Hiroshi Matsuda, Chunpeng Ma, and Masayuki Asahara. Step-by-step instructions and a simple tabular output format improve the dependency parsing accuracy of LLMs. In *Proceedings of IWPT (SyntaxFest 2025)*, pages 11–19. Association for Computational Linguistics, 2025. URL: https://aclanthology.org/2025.iwpt-1.2/.

[26] Hiroshi Matsuda and Masayuki Asahara. Probing the dynamics of syntactic ability acquisition throughout LLM pretraining. In *Proceedings of the Ninth Workshop on Universal Dependencies (UDW 2026)*, pages 1–48. ELRA Language Resources Association (ELRA), 2026. URL: https://aclanthology.org/2026.udw-1.1/, doi:10.63317/2n8gkp49p3wg.

[27] Fabio Tamburini. (better than) state-of-the-art PoS-tagging for italian texts. In *Proceedings of the Third Italian Conference on Computational Linguistics (CLiC-it 2016)*, pages 294–298, 2016. URL: https://aclanthology.org/2016.clicit-1.55/.